
Smart Contract Security Audit Report

1inch SwapVM

An independent security assessment of the 1inch smart contract system.

CUSTOMER

1inch

DATE

2026-07-28 to 2026-07-29

VERSION

1.1

Contents

02	General Information	Intro · Scope · Scoring · Disclaimer
03	Summary	Discovered weaknesses
04	General Recommendations	Security process improvement
05	Findings	1 issues
06	Appendix	About us

General Information

This report contains information about the results of the security audit of 1inch (hereafter referred to as the "Customer") smart contracts, conducted by [Decurity](#) in the period from 2026-07-28 to 2026-07-29.

2.1 Introduction

Tasks solved during the work are:

- Review the protocol design and the usage of 3rd party dependencies,
- Audit the contracts implementation,
- Develop the recommendations and suggestions to improve the security of the contracts.

2.2 Scope of Work

The audit scope included the contracts in the following repository: <https://github.com/1inch/swap-vm>. Initial review was done for the release <https://github.com/1inch/swap-vm/releases/tag/v1.0.2> (commit 32c687c2b73101fc26549e48fa1ff8a4d73afbac)

The following contracts have been tested:

- `src/instructions/Fee.sol`

2.3 Threat Model

The assessment presumes actions of an intruder who might have capabilities of any role (an external user, token owner, token service owner, a contract). The centralization risks have not been considered upon the request of the Customer.

The main possible threat actors are:

- Maker,
- Taker.

2.4 Weakness Scoring

An expert evaluation scores the findings in this report. The impact of each vulnerability is calculated based on its ease of exploitation and its severity for the considered threats.

CRITICAL	Trivially exploitable with catastrophic outcome — total loss of funds, unauthorized control, or permanent loss of contract functionality.
HIGH	Likely exploited and capable of causing significant financial loss or severe operational disruption.
MEDIUM	Exploitable under specific conditions, resulting in moderate harm — operational disruption or limited financial impact.
LOW	Low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
INFO	No immediate security risk; relevant to best practices, code quality, or potential optimizations.

2.5 Disclaimer

Due to the intrinsic nature of software and vulnerabilities, it cannot be generally guaranteed that a certain security property of a program holds.

Therefore, this report is provided "as is" and is not a guarantee that the analyzed system does not contain any other security weaknesses. Furthermore, this report is not an endorsement of the project, nor is it investment advice.

Summary

As a result of this work, we have discovered a single informational issue.

The table below contains the discovered issues, their risk level, and their current status.

CRITICAL 0	HIGH 0	MEDIUM 0	LOW 0	INFO 1
---------------	-----------	-------------	----------	-----------

3.1 Discovered Weaknesses

ISSUE	CONTRACT	RISK LEVEL	STATUS
Creating an instruction with feeBps == BPS results in revert	src/instructions/Fee.sol	INFO	ACKNOWLEDGED

General Recommendations

This section contains general recommendations on how to improve the overall security level.

4.1 Security Process Improvement

- Keep the documentation updated to make it consistent with the implementation,
- Perform regular audits for all new contracts and updates,
- Ensure the secure off-chain storage and processing of privileged private keys,
- Launch a public bug bounty campaign for the contracts.

Findings

5.1

Creating an instruction with feeBps == BPS results in revert

INFO

ACKNOWLEDGED

Client's comment: Design choice, the maker is responsible for creating a strategy with the right settings.

Contracts

src/instructions/Fee.sol

Location

buildFlatFee, buildProtocolFee, _feeAmountIn, _flatFeeAmountInXD

Description

`feeBps` can be set as high as `BPS` in `buildFlatFee` or `buildProtocolFee`. However, in that case, `_feeAmountIn` or `_flatFeeAmountInXD` will revert during a `!isExactIn` calculation branch because of division by zero. For example:

```
ctx.swap.amountIn += Math.ceilDiv(ctx.swap.amountIn * feeBps, BPS - feeBps);
```

Remediation

Consider limiting `feeBps` to be strictly lower than `BPS`.

Appendix

6.1 About us

The Decurity team consists of experienced hackers who have been doing application security assessments and penetration testing for over a decade.

During recent years, we've gained expertise in the blockchain field and have conducted numerous audits for both centralized and decentralized projects: exchanges, protocols, and blockchain nodes.

Our efforts have helped to protect hundreds of millions of dollars and make web3 a safer place.

